RED BLACK TREES

RB-Trees

Red Black Tree is a type of **self balancing** binary search tree

- It is invented in 1972 by Rudolf Bayer who called it "symmetric binary B-Trees"
- Each node of this tree has an extra attribute 'color' which can be RED or BLACK
- Coloring of nodes ensures that longest path from root to a leaf is no longer then twice the length of a shortest path which means the tree is balanced

AVL Vs RB Tree

Parameter	AVL Tree	R-B tree
Balance Factor	**AVL trees are more strictly balanced compared to RB trees.** In AVL trees, the balance factor (the height difference between the left and right subtrees) of every node is restricted to be at most 1, ensuring a height-balanced tree.	**RB trees have less strict balance requirements.** They ensure that the tree remains approximately balanced by coloring the nodes (red or black) and enforcing several properties.
Insertion and Deletion	Insertions and deletions in AVL trees typically require more rotations, which can make them slower for these operations. Balancing an AVL tree can take multiple rotations in some cases.	RB trees require fewer rotations during insertion and deletion, making them more efficient for dynamic datasets where elements are frequently added or removed. RB trees are considered more practical for use in databases and file systems where there are many insertions and deletions.

AVL Vs RB Tree

Parameter	AVL Tree	R-B tree
Performance Considerations	AVL trees are well-suited for scenarios where read operations significantly outnumber write operations, as their read performance is generally better due to their stricter balance criteria.	RB trees are preferred when the dataset is more dynamic, and there are frequent insertions and deletions. Their performance is often better in situations with a mix of read and write operations, and they are commonly used in real-time systems.
Code Complexity	AVL trees tend to have more complex code for maintaining balance, which can lead to larger codebases and potentially more room for bugs.	RB trees are usually easier to implement, maintain, and debug due to their simpler balance maintenance.

AVL *Vs* RB *Vs* B-Tree

	Red-Black Tree	AVL Trees	B-Tree
Tree Type	Self-balancing binary search tree	Self-balancing binary search tree	Multi-way tree(N – ary tree).
Child Number	Every node contains at most 2 child nodes	Every node contains at most 2 child nodes	Nodes can have multiple child nodes
Self-Balancing Methods	Same as AVL, RB trees possess rotation capabilities for self-balancing. However, before rotating, red-black trees change the color of the nodes from red to black or vice-versa to self-balance the depth of the tree.	It has a balance factor whose value is either -1, 0, or 1. Balance factor = (height of left subtree)-(height of right subtree) or Balance factor = (height of right subtree)-(height of left subtree)	B-Tree is defined by the term minimum degree 't'. The value of 't' depends upon disk block size. Every node except the root must contain at least t-1 keys. The root may contain a minimum of 1 key.
Tree Height	RB tree has a height of log(N), where N represents the depth of the RB tree in terms of black nodes	AVL tree has a height of log(N) (Where N is the number of nodes).	B-tree has a height of log(M*N) (Where 'M' is the order of the tree and N is the number of nodes).

The height (h) of a B-tree is given by:

$$h \leq \left\lceil \log_{\lceil m/2 \rceil} \frac{n+1}{2} \right\rceil - 1$$

Properties of RB Tree

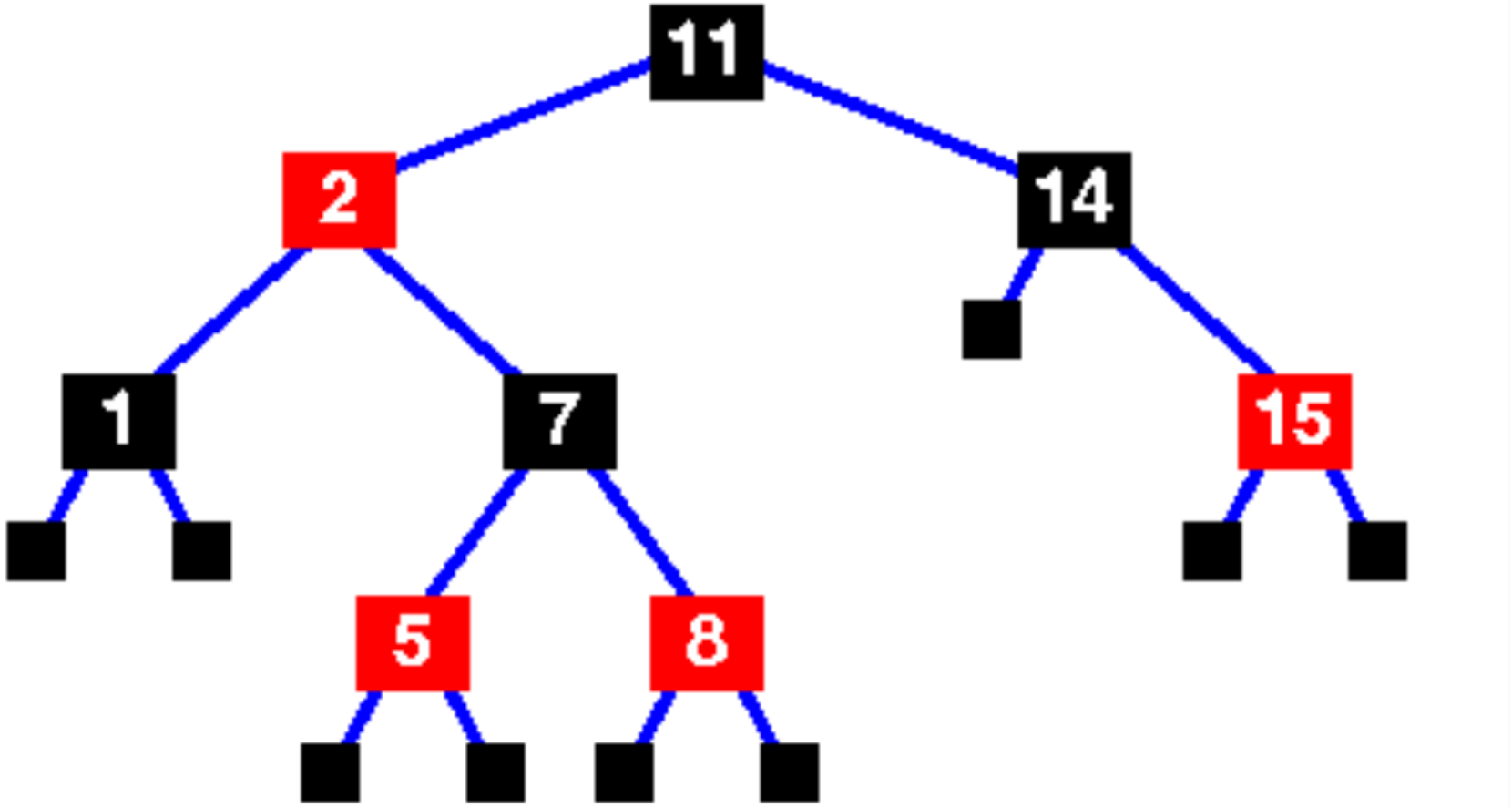
1. ***Root*** is always black

2. ***Nil*** is considered to be black which means that every non-Nil node has two children

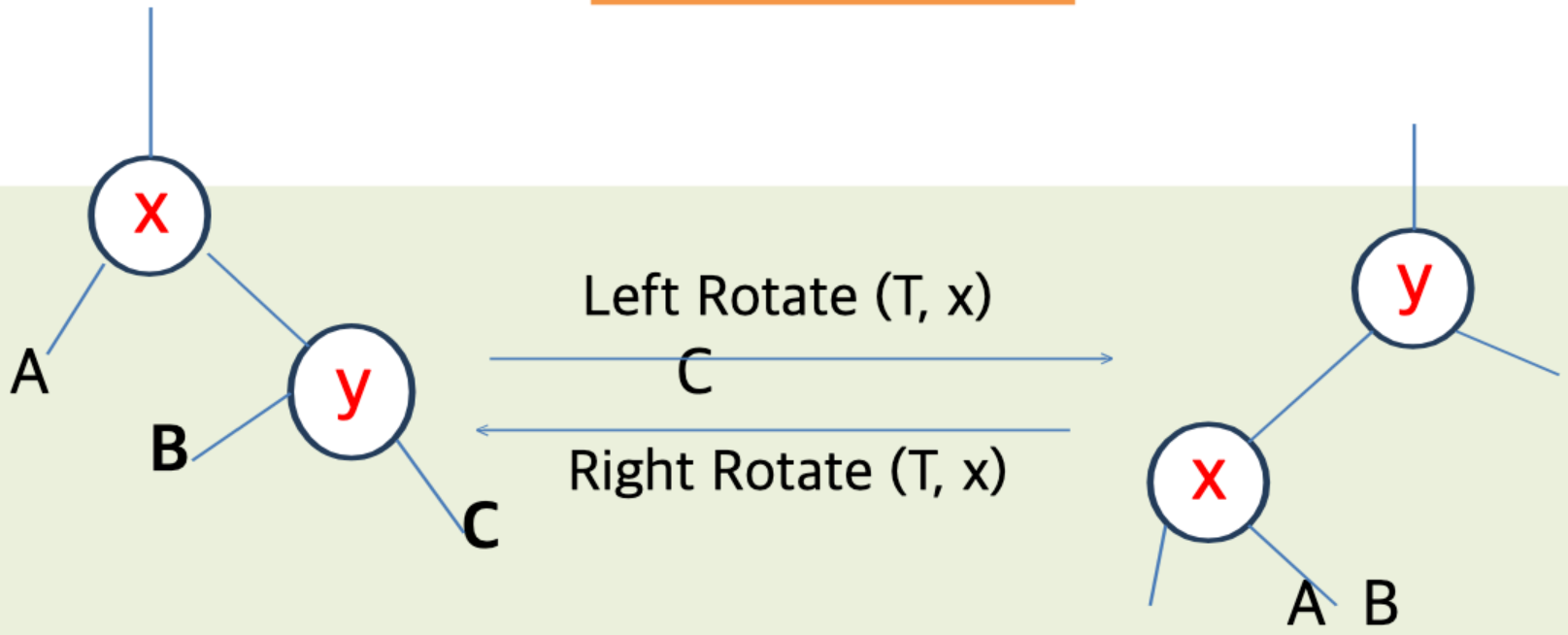
3. ***Black Children Rule***: children of each red node are black

4. ***Black Height Rule:*** For each node v, there exist an integer ***bh***(v) such that each downward path from v to a nil has exactly bh(v) black real(ie non nill) nodes.

Ex- RB Tree



Rotations



The rotations do not break BST- property
The order (AxByC is preserved)

Insertion

- Insert (New Node Must be Red)
- Arrange Color according to RBT Properties
  - Root is always Black
  - Every leaf which is NIL is black
  - If node is red then its both children must be black or If a node is red, then its parent must be black.
- Rotation

Violation of properties of red black tree

- If New node is Root node
- If Parent of New Node is Red

The type of discrepancy is determined by

➔the location of the node with respect to its grand parent, and

➔ the color of the sibling of the parent

Discrepancies

1. ***where the parent sibling (uncle) is red*** :

fixed by changing color

2 ***where the parent sibling (uncle) is black*** :

fixed by rotations

(note: at most one rotation is sufficient for fixing the problem)

RB-Insertion$_{1/2}$

- Start by doing regular binary-search-tree insertion.

```
RB-INSERT(T, z)
1.   y ← NIL; x ← root[T]
2.   while x ≠ NIL
3.        do y ← x
4.           if key[z] < key[x]
5.              then x ← left[x]
6.              else x ← right[x]
7.   p[z] ← y
8.   if y = NIL
9.      then root[T] ← z        /* Tree T was empty */
10.     else if key[z] < key[y]
11.             then left[y] ← z
12.             else right[y] ← z
13.  left[z] ← nil[T]; right[z] ← nil[T]; color[z] ← RED
14.  RB-INSERT-FIXUP(T, z)
```



RB-Insertion$_{1/2}$

- Start by doing regular binary-search-tree insertion.

```
RB-INSERT(T, z)
1.   y ← NIL; x ← root[T]
2.   while x ≠ NIL
3.         do y ← x
4.            if key[z] < key[x]
5.               then x ← left[x]
6.               else x ← right[x]
7.   p[z] ← y
8.   if y = NIL
9.      then root[T] ← z        /* Tree T was empty */
10.     else if key[z] < key[y]
11.             then left[y] ← z
12.             else right[y] ← z
13.  left[z] ← nil[T]; right[z] ← nil[T]; color[z] ← RED
14.  RB-INSERT-FIXUP(T, z)
```

RB-Insertion$_{2/2}$

- RB-INSERT ends by coloring the new node z red.
- Then it calls RB-INSERT-FIXUP because we could have violated a red-black property.
- Which property might be violated?
  1. If z is the root, then there's a violation. Otherwise, OK.
  2. If $p[z]$ is red, there's a violation: both z and $p[z]$ are red.



RB-INSERT-FIXUP procedure

```
RB-INSERT-FIXUP(T, z)
1.  while color[p[z]] = RED
2.      do if p[z] = left[p[p[z]]]
3.            then y ← right[p[p[z]]]
4.                 if color[y] = RED
5.                    then color[p[z]] ← BLACK              Case 1
6.                         color[y] ← BLACK                Case 1
7.                         color[p[p[z]]] ← RED            Case 1
8.                         z ← p[p[z]]                     Case 1
9.                    else if z = right[p[z]]
10.                             then z ← p[z]              Case 2
11.                                  LEFT-ROTATE(T, z)     Case 2
12.                        color[p[z]] ← BLACK             Case 3
13.                        color[p[p[z]]] ← RED            Case 3
14.                        RIGHT-ROTATE(T, p[p[z]])        Case 3
15.           else (same as then clause
16.                with "right" and "left" exchanged)
17. color[root[T]] ← BLACK
```

Time complexity of RB-INSERT

- Analysis:
  - Each iteration takes $O(1)$ time.
  - The **while** loop repeats only if case 1 is executed, and then the pointer z moves two levels up the tree.
  - The **while** loop terminates if case 2 or case 3 is executed.
  - $O(\lg n)$ levels → $O(\lg n)$ time.
  - Also note that there are at most 2 rotations overall.

- Thus, insertion into a red-black tree takes $O(\lg n)$ time.

Understanding Insertion

Let x be the newly inserted node.

1) Perform standard BST insertion and make the color of newly inserted nodes as RED.

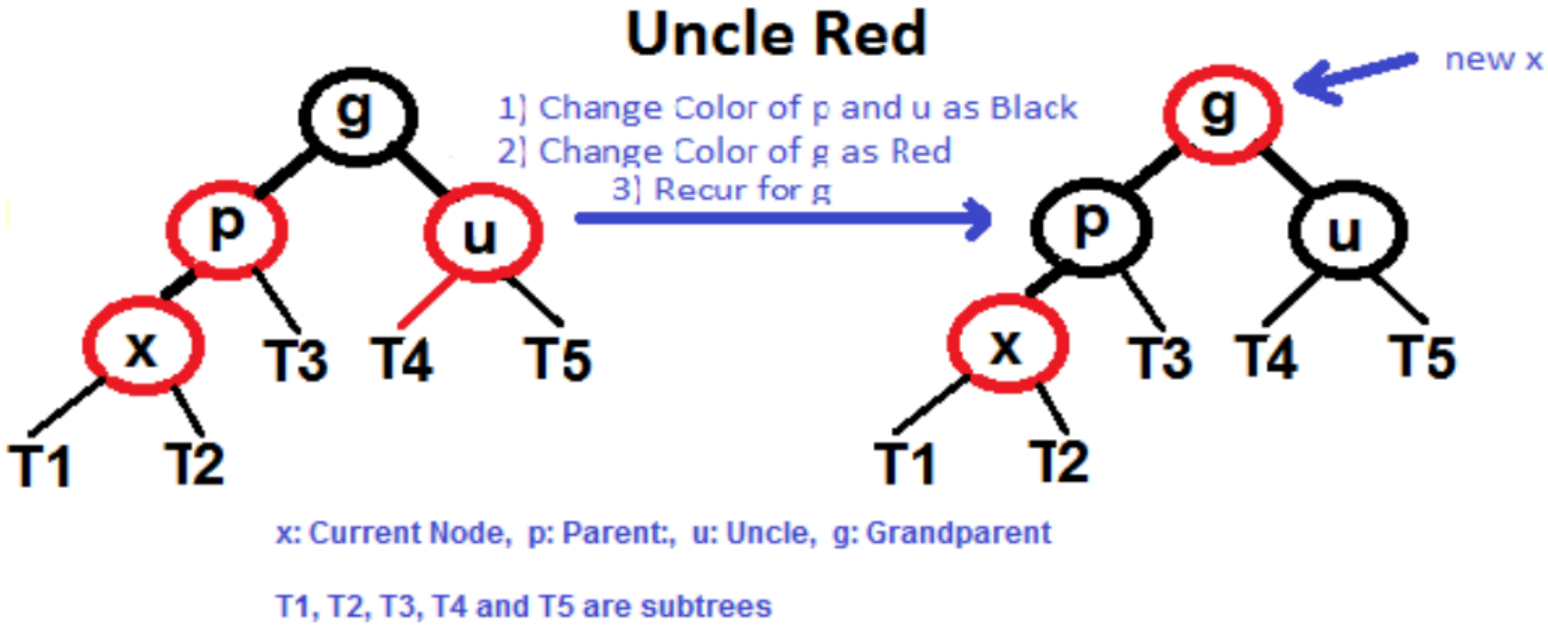
2) If x is root, change color of x as BLACK

3) Do following if color of x's parent is not BLACK or x is not root.

a) If x's uncle is RED (*case1*)

(i) Change color of parent and uncle as BLACK.

(ii) color of grand parent as RED.



b) If x's uncle is BLACK,
then there can be four configurations for x, x's parent (**p**) and x's grandparent (**g**)

i) Left Left Case (p is left child of g and x is left child of p)

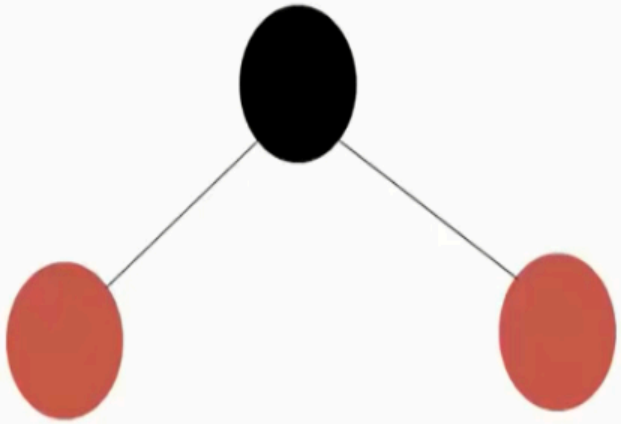
ii) Left Right Case (p is left child of g and x is right child of p)

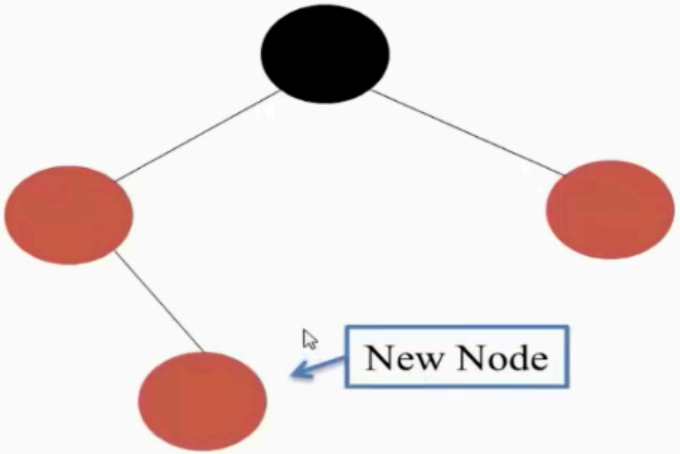
iii) Right Right Case (Mirror case i)

iv) Right Left Case (Mirror case ii)

Inserting a new node

Case 1: New Node's Uncle is Red



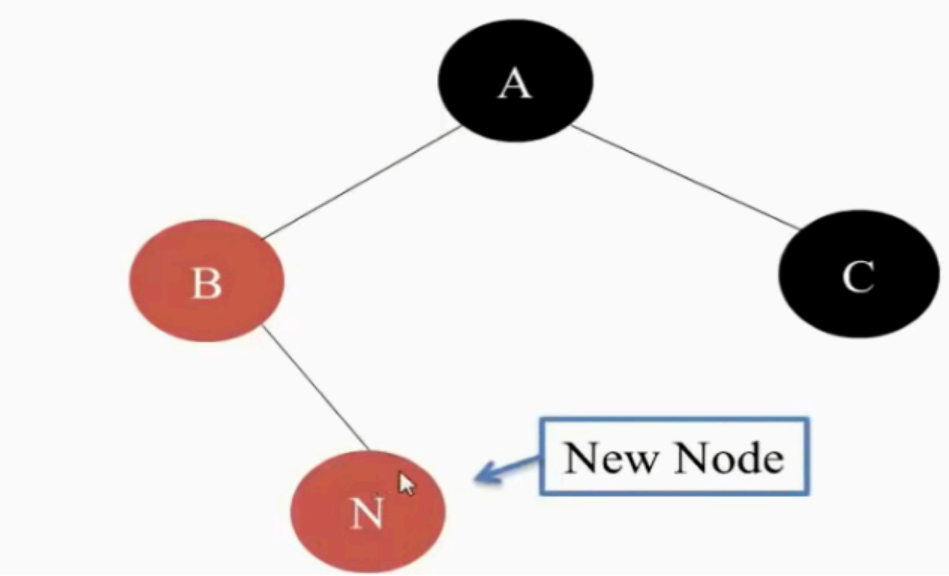


Strategy to resolve :

CHANGE COLOR

1. Change Color of Grand Parent of New Node
2. Change Color of Parent and Uncle of New Node
3. Grandparent will be marked as New Node

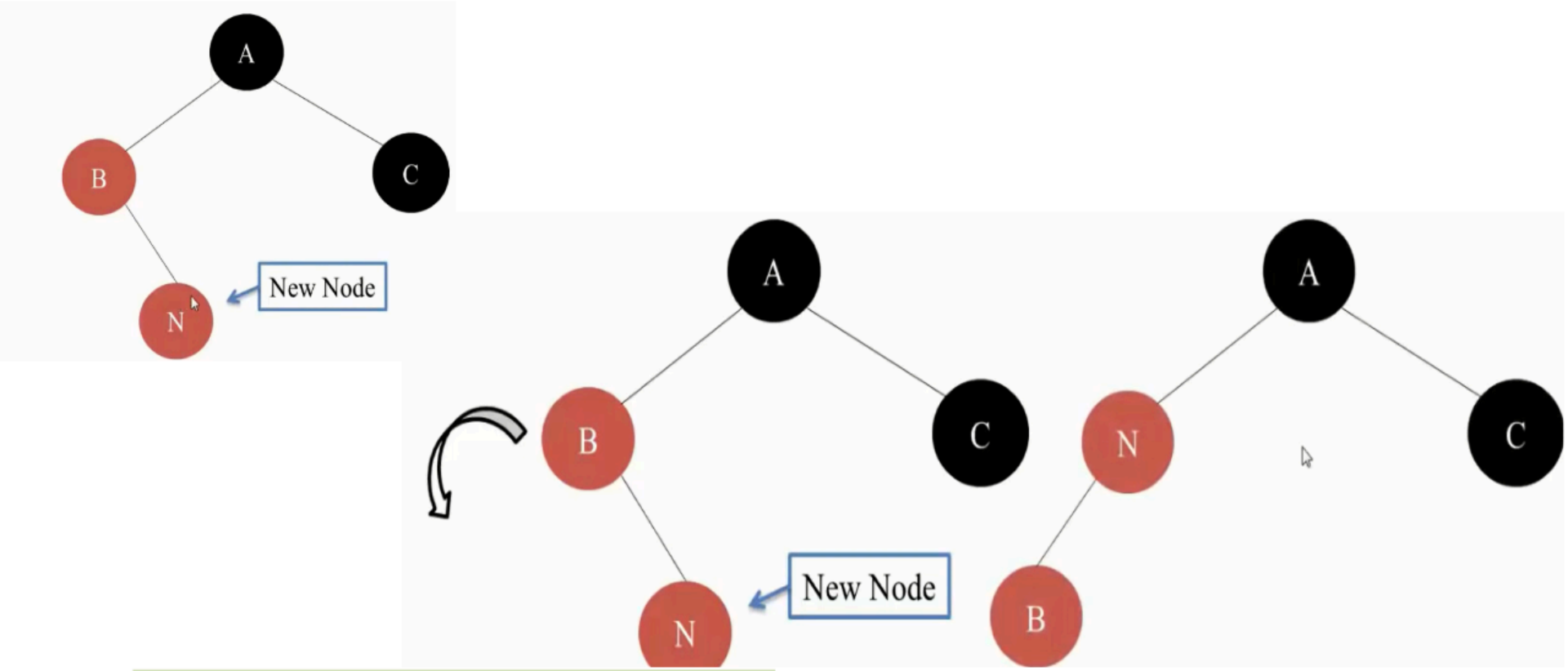
Case 2: New Node's Uncle is Black and New Node is right Child of its parent



Strategy to resolve :
ROTATION

- ✓Anti Clockwise Rotation at Parent Node
- ✓Make Parent as New Node

Case 2: New Node's Uncle is Black and New Node is right Child of its parent

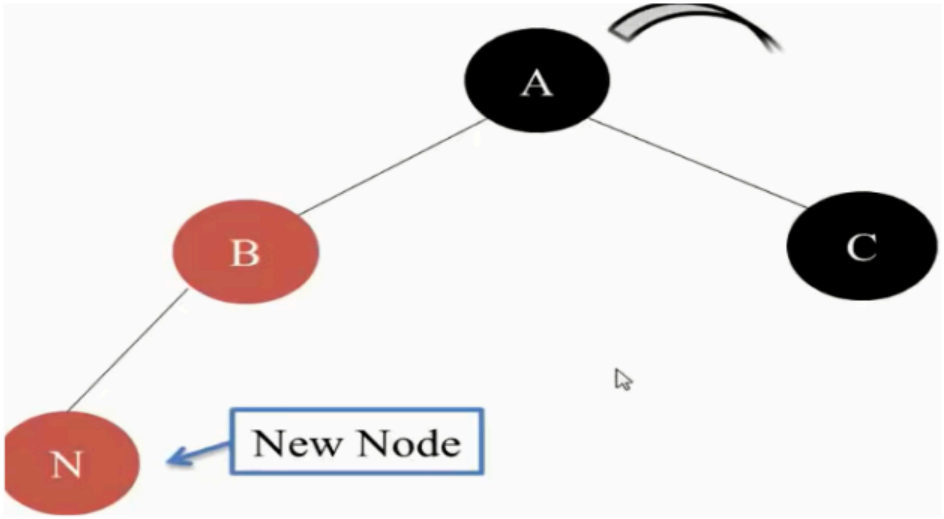


Strategy to resolve :

ROTATION

Anti Clockwise Rotation at Parent Node
Make Parent as New Node

Case 3 New Node's Uncle is black and New Node is left Child of its Parent



Strategy to resolve :

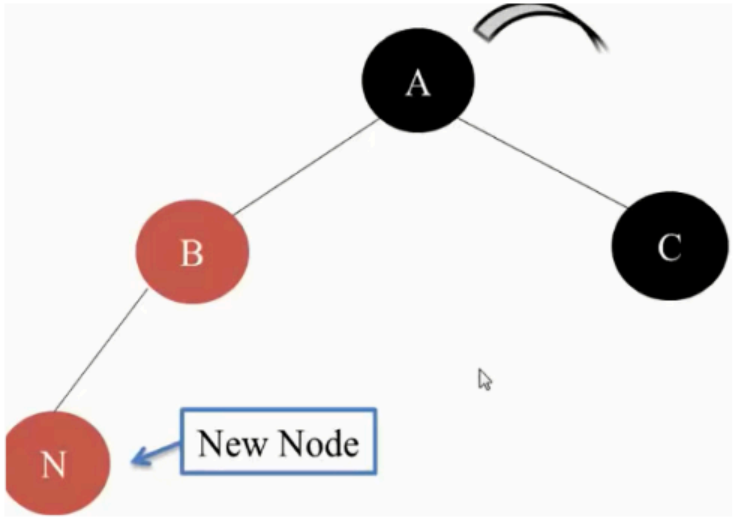
ROTATION & COLOR CHANGE

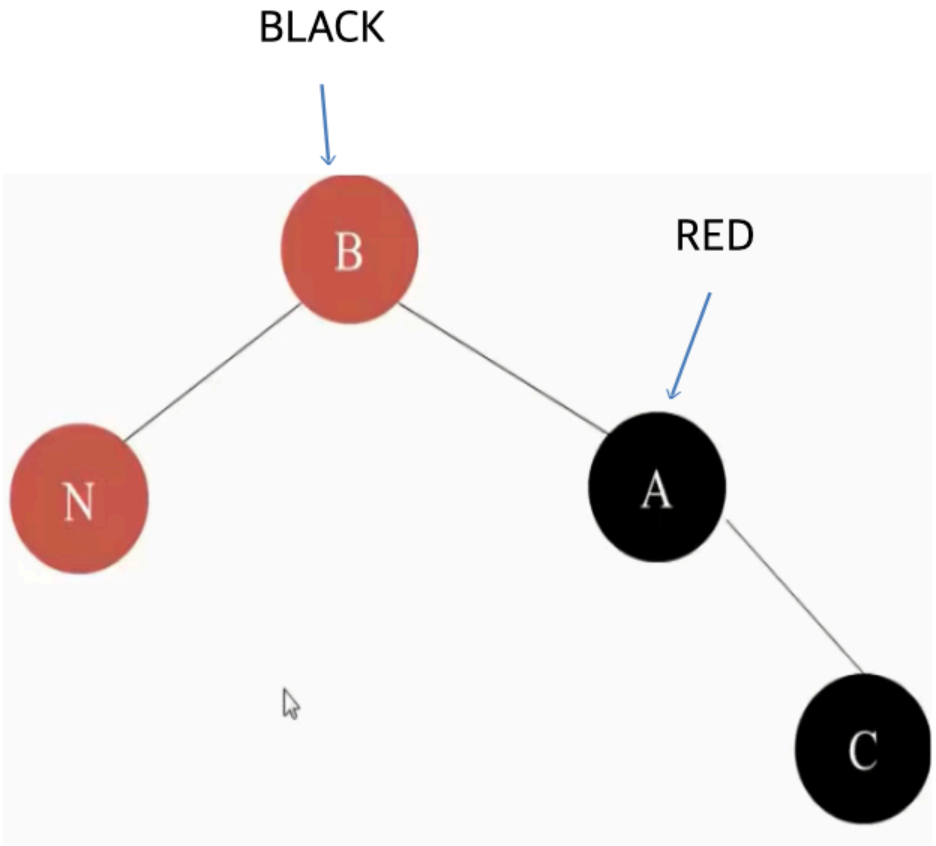
✓Clockwise Rotation at GrandParent Node

✓Exchange Color of GrandParent and Parent

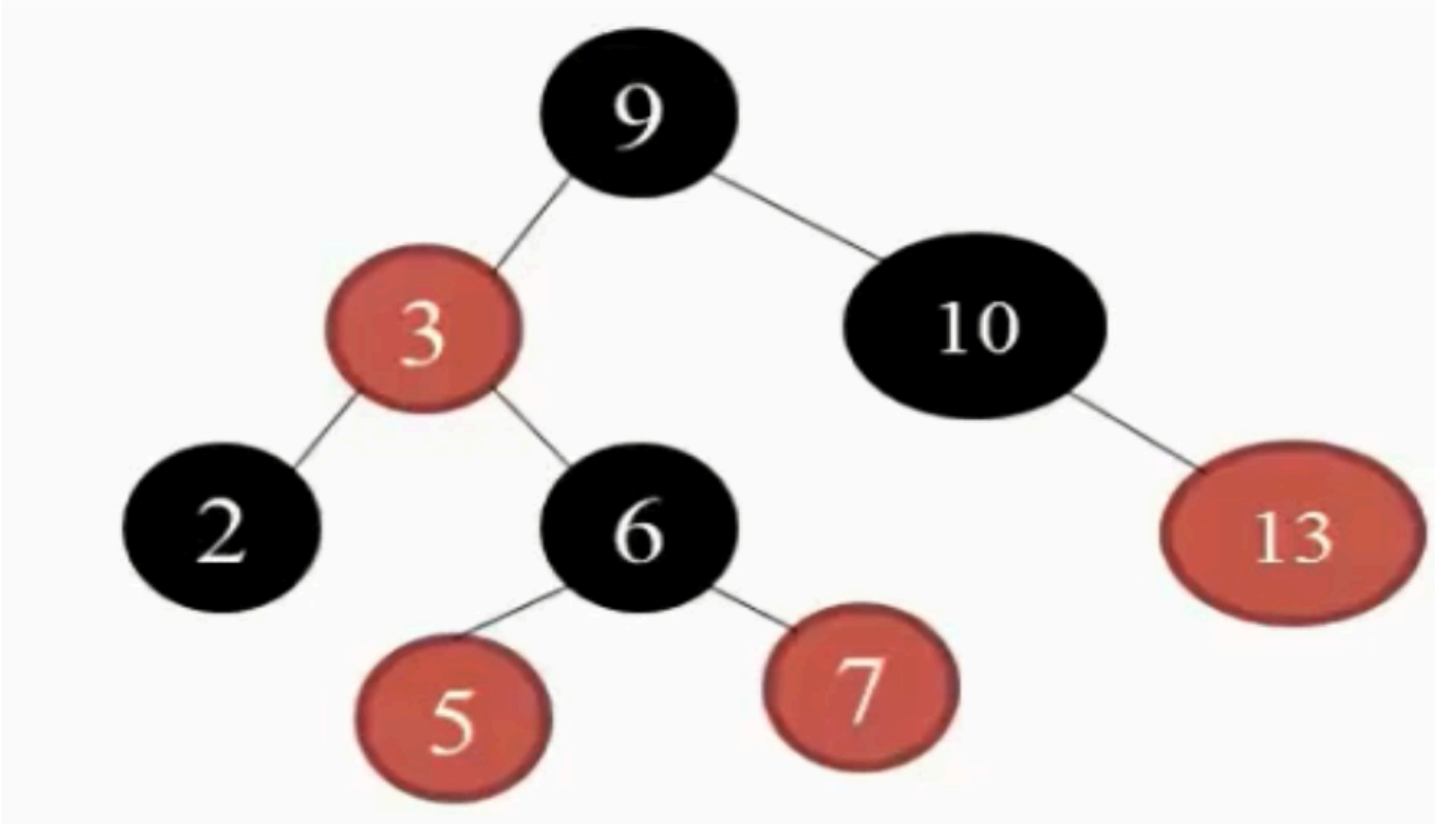
Case 3

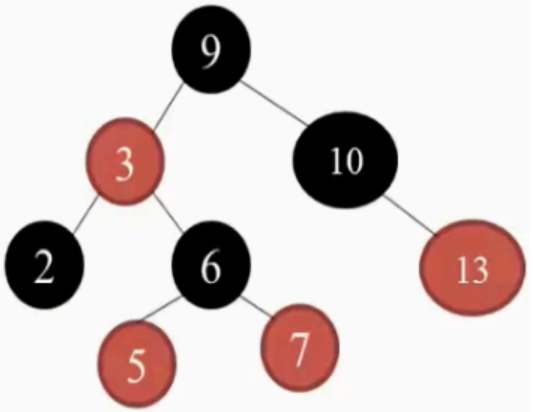
1. Clockwise Rotation in grand Parent of New Node
2. Exchange the color of Grand Parent and Parent of New Node





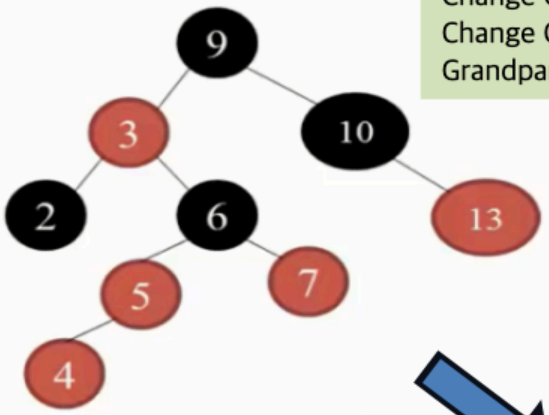
Example



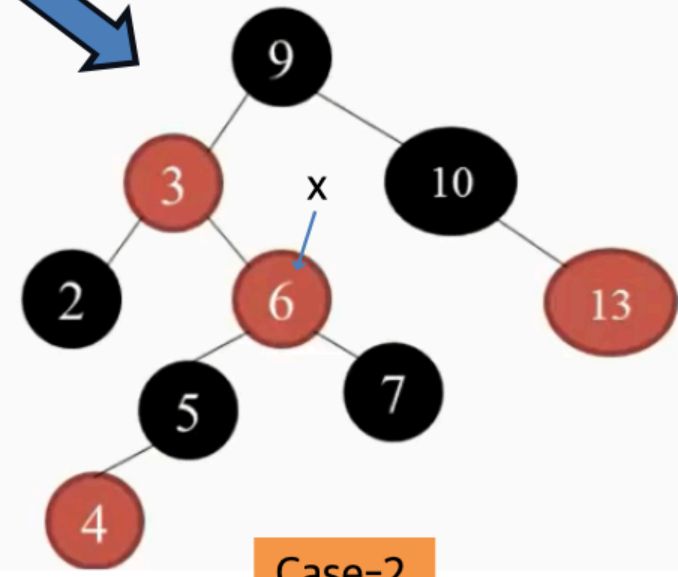


Insert 4

Change Color of Grand Parent of New Node
Change Color of Parent and Uncle of New Node
Grandparent will be marked as New Node

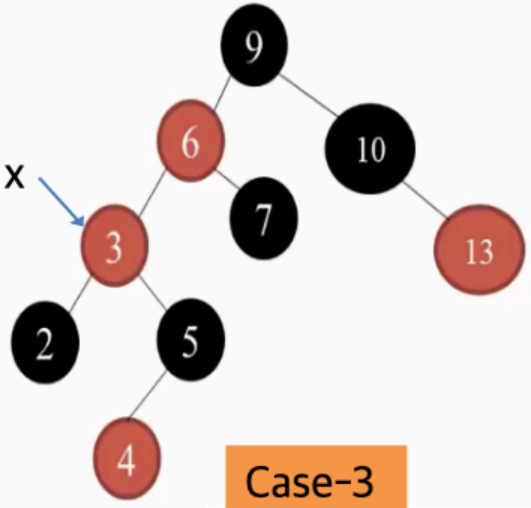


Case-1



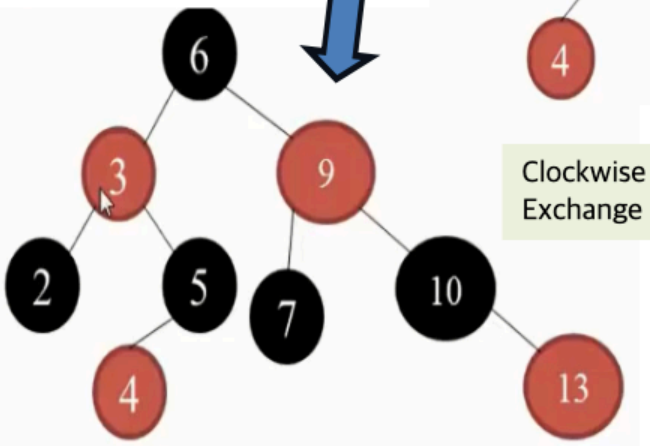
Case-2

Anti Clockwise Rotation at Parent Node
Make Parent as New Node



Case-3

Clockwise Rotation at GrandParent Node
Exchange Color of GrandParent and Parent



Insert following nodes in an empty RB Tree

- 41, 38,31, 12, 19, 8

- 5, 10, 15, 25, 20, 30

Exercises

-1 Draw the complete binary search tree of height 3 on the keys {1, 2, . . . , 15}. Add the NIL leaves and color the nodes in three different ways such that the black-heights of the resulting red-black trees are 2, 3, and 4.

-2 Suppose that the root of a red-black tree is red. If we make it black, does the tree remain a red-black tree?

Video Lecture of the topic can be found at

Red Black Tree and Insertion

https://www.youtube.com/watch?v=H0MutN9u-Ik

Understanding RB Tree Deletion cases with an Example

https://youtu.be/1qEjtfR0gIo

How To Perform Search & Insert Operation In Binary Search Tree (BST)

https://www.youtube.com/watch?v=smE0gHcd5Gg